

Fehlermelde-Modul einbauen

Anleitung für eine Claude-Sitzung, die eine Flutter-App um Fehlerberichte erweitern soll

Worum es geht

Das Modul ist eine einzige Dart-Datei. Eingebaut blendet es auf jedem Bildschirm einen kleinen Fehler-Button ein, fängt Abstürze selbsttätig ab und legt jeden Bericht samt Protokoll und Bildschirmfoto in Notion ab. Es braucht nur zwei Pakete: **http** und **url_launcher**.

In Notion ist nichts vorzubereiten. Beim ersten Bericht sucht sich die App über ihren App-Key in der Registry **Apps** unter der Seite **Fehlerzentrale**. Findet sie sich dort nicht, legt sie sich selbst eine Unterseite mit eigener Datenbank an und trägt sich ein.

Schritt 1 - Dateien holen

Liegt das Repo **Flai331/Programmieren** vor, sind die Dateien schon da, unter `shared/fehlerbericht/`. Andernfalls herunterladen - das Repo ist öffentlich lesbar:

```
REPO=https://raw.githubusercontent.com/Flai331/Programmieren
ZWEIG=claude/fehlerberichte-programm-cleanup-1cvy8n
BASE=$REPO/$ZWEIG/shared/fehlerbericht

curl -sSO $BASE/tools/einbauen.py
curl -sSO $BASE/tools/app_pruefen.py
mkdir -p lib
curl -sS -o lib/fehlerbericht.dart $BASE/lib/fehlerbericht.dart
```

Wichtig ist die Ablage: `einbauen.py` sucht die Moduldatei relativ zu sich selbst unter `../lib/fehlerbericht.dart`. Die Befehle oben legen genau diese Struktur an.

Schritt 2 - Einbauen

Ein Befehl erledigt alles Mechanische:

```
python3 einbauen.py <app-ordner> --key <schluessel> --name "<App-Name>"
```

Das Skript

- kopiert `lib/fehlerbericht.dart` in die App,
- trägt `http` und `url_launcher` in die `pubspec.yaml` ein,
- setzt die `INTERNET`-Berechtigung ins Android-Manifest,
- verdrahtet `lib/main.dart: Import, Fehlerbericht.runApp` und die drei Parameter an der `MaterialApp`.

Es ist gefahrlos wiederholbar und legt vor jeder Änderung eine `.bak`-Kopie an. Zeilen, die es mit ! ausweist, brauchen Handarbeit - dort steht jeweils, was zu tun ist.

Schritt 3 - Prüfen

```
flutter pub get
python3 app_pruefen.py <app-ordner>
flutter analyze
```

Das Prüfskript muss mit Rückgabewert **0** enden und `flutter analyze` ohne Befund durchlaufen. Erst dann ist der Einbau fertig.

Schritt 4 - Bauen

Das Notion-Token wird beim Bauen gesetzt, niemals in den Quellcode geschrieben. Ein Token gilt für alle Apps.

```
flutter build apk --release --dart-define=NOTION_TOKEN=<token>
```

Wer das Token nicht hat, baut einfach ohne. Die App läuft dann völlig normal und öffnet für Berichte die E-Mail-App statt Notion. Das Token gehört dem Besitzer des Notion-Bereichs und muss nicht in der einbauenden Sitzung bekannt sein.

Falls der Einbau von Hand nötig ist

Es sind genau vier Dinge.

1. Moduldatei

`fehlerbericht.dart` unverändert nach `lib/` der App kopieren.

2. pubspec.yaml

```
dependencies:
  http: ^1.2.0
  url_launcher: ^6.2.5
```

3. Android-Manifest

In `android/app/src/main/AndroidManifest.xml` direkt unter `<manifest ...>`:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Nicht überspringen, auch wenn die Zeile schon in `src/debug/` steht: Release-Builds erben sie von dort nicht. Ohne sie scheitert jeder Notion-Aufruf mit `Failed host lookup ... errno = 7`, und die App weicht still auf E-Mail aus. Das ist die häufigste Stolperfalle überhaupt, weil im Debug-Betrieb alles zu funktionieren scheint.

4. main.dart

```
import 'fehlerbericht.dart';

void main() => Fehlerbericht.runApp(
  appKey: 'meine_app',
  appName: 'Meine App',
  version: '1.0.0',
  builder: () => const MyApp(),
);
```

und an der MaterialApp:

```
MaterialApp(
  navigatorKey: Fehlerbericht.navigatorKey,
  navigatorObservers: [Fehlerbericht.observer],
  builder: Fehlerbericht.wrap,
  // alles Übrige unverändert
)
```

Regeln, die nicht verhandelbar sind

Regel	Warum
Der App-Key ist klein geschrieben, ohne Leerzeichen, und ändert sich nie mehr.	Er ist der Schlüssel in der Notion-Registry. Wird er geändert, legt die App eine zweite Datenbank an und die bisherigen Berichte sind abgehängt.
Das Token steht nie im Quellcode, nur in <code>--dart-define</code> .	Sonst landet es im Repo und in jeder ausgelieferten App.
Die Moduldatei wird nicht angepasst.	Sie ist in allen Apps identisch. Wer sie ändert, verliert künftige Verbesserungen; das Prüfskript meldet Abweichungen.
Auf Web ist der E-Mail-Weg das erwartete Verhalten.	Notion beantwortet keine Anfragen aus dem Browser (CORS). Das Modul erkennt das und wechselt selbsttätig.

Wenn kein Bericht in Notion ankommt

Das Protokoll nennt die Ursache im Klartext - es steht im geöffneten E-Mail-Entwurf und in der Debug-Ausgabe:

Meldung im Protokoll	Ursache
Kein NOTION_TOKEN gesetzt	Beim Bauen fehlte <code>--dart-define</code> .
Failed host lookup ... errno = 7	INTERNET-Berechtigung fehlt im main-Manifest (siehe Punkt 3).
Registry-Abfrage HTTP 401	Token falsch oder abgelaufen.
HTTP 404	Die Integration hat keinen Zugriff auf die Fehlerzentrale. Der Besitzer teilt die Seite einmalig über ... -> Verbindungen.
CORS-Blockade / Web nicht erreichbar	Erwartet im Browser, siehe oben.

Fertig ist der Einbau, wenn

- `app_pruefen.py` mit Rückgabewert 0 endet,
- `flutter analyze` keinen Befund meldet,
- die App startet und unten rechts der Fehler-Button sichtbar ist,
- ein abgeschickter Testbericht in Notion unter der Fehlerzentrale auftaucht (nur mit Token prüfbar).

Ausführliche Fassung samt vollständiger API: `shared/fehlerbericht/README.md` und `EINBAU.md` im selben Ordner.